

Kubernetes

Formation 2 jours – Jour 2

**Administration, Enrichissement de clusters, Service Mesh Istio,
Écosystème**

Administration de Kubernetes

Opérer un cluster au quotidien

Déploiement d'application – stratégies

Stratégie	Description	Cas d'usage
Recreate	Tout supprimer puis tout recréer (downtime)	Migrations schéma incompatibles
RollingUpdate	Mise à jour progressive pod par pod (par défaut)	Cas général, zéro downtime
Blue/Green	Deux versions complètes en parallèle, bascule atomique	Tests en prod avant bascule
Canary	Petit % de trafic vers la nouvelle version	Validation progressive en prod
Shadow	Trafic dupliqué vers la nouvelle version (lecture seule)	Tests de charge en conditions réelles

```
# RollingUpdate avec contraintes strictes
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%           # % de pods supplémentaires pendant l'update
      maxUnavailable: 0      # zéro pod down
```

“ Les stratégies avancées (Blue/Green, Canary) s'implémentent avec un Service Mesh (Istio) ou un controller dédié (Argo Rollouts, Flagger). ”

Blue/Green à la main

```
# 1. Le Deployment "blue" tourne déjà
kubectl get deploy app-blue
kubectl get svc app    # selector: { version: blue }

# 2. Déployer "green" à côté
kubectl apply -f deployment-green.yaml    # selector différent : version=green

# 3. Tests internes sur green
kubectl port-forward deploy/app-green 8080:80
curl localhost:8080/health

# 4. Bascule atomique : modifier le selector du Service
kubectl patch svc app -p '{"spec":{"selector":{"version":"green"}}}'

# 5. Retour arrière instantané si problème
kubectl patch svc app -p '{"spec":{"selector":{"version":"blue"}}}'

# 6. Supprimer "blue" une fois green stable
kubectl delete deploy app-blue
```

“ Coût : 2x les ressources pendant la bascule. Avantage : rollback en 1 seconde.

”

Canary à la main

Idée : deux Deployments partagent le même label de selector du Service, ratio contrôlé par le nombre de réplicas.

```
# stable (9 pods sur 10)
metadata: { name: app-stable, labels: { app: web, track: stable } }
spec:
  replicas: 9
  template:
    metadata: { labels: { app: web } }      # label commun
---
# canary (1 pod sur 10 – donc ~10% du trafic)
metadata: { name: app-canary, labels: { app: web, track: canary } }
spec:
  replicas: 1
  template:
    metadata: { labels: { app: web } }      # même label commun
```

Canary à la main

```
# Service ne sélectionne que app=web → load balance sur les 10 pods
apiVersion: v1
kind: Service
spec:
  selector: { app: web }
```

“ Pour un canary par **% de trafic** (et non par nombre de pods), utilisez Istio ou Argo Rollouts. ”

Gestion des ressources – ResourceQuota

```
apiVersion: v1
kind: ResourceQuota
metadata: { name: dev-quota, namespace: dev }
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    persistentvolumeclaims: "10"
    pods: "50"
    services.loadbalancers: "2"
    count/deployments.apps: "20"
```

Gestion des ressources — ResourceQuota

```
kubectl describe quota -n dev  
kubectl get resourcequota -A
```

“ Une fois la quota atteinte, **toute création** dépassant la limite est rejetée par l'admission controller `ResourceQuota`. ”

Gestion des ressources – LimitRange

Force des valeurs par défaut et des bornes au sein d'un namespace.

```
apiVersion: v1
kind: LimitRange
metadata: { name: dev-limits, namespace: dev }
spec:
  limits:
    - type: Container
      default:                # appliqué si non spécifié
        cpu: 200m
        memory: 256Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:                    # limite maximale
        cpu: "2"
        memory: 2Gi
      min:                    # limite minimale
        cpu: 10m
        memory: 16Mi
```

“ Très utile pour éviter qu'un dev déploie un pod sans aucune limite ou avec une limite déraisonnable. ”

PriorityClass et PodDisruptionBudget

PriorityClass : priorité d'éviction quand le cluster est saturé.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high-priority }
value: 1000000
globalDefault: false
description: "Critical workloads"
---
# Utilisation dans un Pod
spec:
  priorityClassName: high-priority
```

PriorityClass et PodDisruptionBudget

PodDisruptionBudget : garantir un minimum de pods disponibles lors des évictions volontaires (drain, upgrade).

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: api-pdb }
spec:
  minAvailable: 2          # ou maxUnavailable: 1
  selector:
    matchLabels: { app: api }
```

Troubleshooting – workflow

Symptôme	Première commande	Investigation
Pod Pending	→ <code>kubectl describe pod</code>	Events, scheduling
Pod CrashLoop	→ <code>kubectl logs --previous</code>	Exit code, stack trace
Pod ImagePull	→ <code>kubectl describe pod</code>	Registry, secret docker
OOMKilled	→ <code>kubectl describe pod</code>	Memory limit trop bas
Service injoign.	→ <code>kubectl get endpoints</code>	Selector ↔ labels
Slow	→ <code>kubectl top pods</code>	CPU throttling
Init stuck	→ <code>kubectl logs -c <init></code>	InitContainers

Troubleshooting — workflow

Boîte à outils :

```
kubectl get events --sort-by=.lastTimestamp -A
kubectl describe pod <pod>
kubectl logs <pod> -c <container> --previous --tail=200
kubectl exec -it <pod> -- /bin/sh
kubectl debug <pod> -it --image=nicolaka/netshoot --target=<container>
kubectl get pods -o wide
kubectl rollout status deploy/<name>
kubectl top pods --containers -A
```

Troubleshooting — `kubectl debug`

`kubectl debug` injecte un **ephemeral container** dans un pod en cours d'exécution (sans redéploiement).

```
# Ouvrir un shell avec des outils réseau dans un pod
kubectl debug -it mon-pod \
  --image=nicolaka/netshoot \
  --target=app
# Copier un pod pour le debugger
kubectl debug mon-pod -it \
  --image=busybox \
  --copy-to=mon-pod-debug \
  --share-processes
# Debug d'un nœud (chroot dans /host)
kubectl debug node/worker-1 -it --image=ubuntu
chroot /host
```

Troubleshooting — `kubectl debug`

Outils utiles dans une image debug :

- `dig`, `nslookup`, `host` — DNS
- `curl`, `wget`, `httpie` — HTTP
- `tcpdump`, `ngrep`, `mtr` — réseau bas niveau
- `nc`, `nmap` — ports ouverts
- `iperf3` — bande passante

Autoscaling – HPA

Horizontal Pod Autoscaler : ajuste le nombre de réplicas selon des métriques.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
    - type: Resource
      resource:
        name: memory
        target: { type: AverageValue, averageValue: 500Mi }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # éviter les flaps
    policies:
      - { type: Percent, value: 10, periodSeconds: 60 }
```

“ Le HPA a besoin du **Metrics Server** installé pour fonctionner.

”

Autoscaling – VPA et Cluster Autoscaler

Vertical Pod Autoscaler (VPA) : ajuste **requests/limits** des Pods automatiquement.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: api-vpa }
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  updatePolicy:
    updateMode: "Auto"          # Off (recommandation seule), Initial, Recreate, Auto
  resourcePolicy:
    containerPolicies:
      - containerName: '*'
        minAllowed: { cpu: 50m, memory: 64Mi }
        maxAllowed: { cpu: 2,   memory: 2Gi }
```

Autoscaling – VPA et Cluster Autoscaler

Cluster Autoscaler : ajoute/retire des **nœuds** quand des pods restent Pending.

- AWS : intégration AutoScalingGroup
- GCP : intégration MIG
- Azure : VMSS
- **Karpenter** (AWS) : alternative moderne, plus rapide

“ VPA et HPA ne doivent **pas** être utilisés sur la même métrique

Bonnes pratiques — Images & Build

- **Tags immuables** : pas de `:latest` en production. Utilisez `:v1.2.3` ou un hash SHA (`@sha256:...`).
- **Images minimales** : `distroless`, `alpine`, `scratch`. Réduit la surface d'attaque et la taille.
- **Multi-stage builds** : compilation dans une image, exécution dans une autre.
- **Non-root** : `USER 1000` dans le Dockerfile. Pair avec `runAsNonRoot: true`.
- **Scan de vulnérabilités** : Trivy, Grype, Snyk en CI.

Bonnes pratiques – Images & Build

```
# Exemple multi-stage minimal
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/api

FROM gcr.io/distroless/static:nonroot
COPY --from=build /app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

Bonnes pratiques – Manifestes & Runtime

- **Toujours définir requests/limits** pour CPU et mémoire.
 - **Probes** : `livenessProbe` (redémarrer), `readinessProbe` (recevoir du trafic), `startupProbe` (démarrage lent).
 - **SecurityContext** : `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`.
 - **PodDisruptionBudget** sur tout workload critique.
 - **NetworkPolicies** par défaut deny-all dans les namespaces sensibles.
 - **Labels standards** : `app.kubernetes.io/name`, `app.kubernetes.io/version`, `app.kubernetes.io/part-of`.
 - **Namespaces séparés** par environnement (dev/staging/prod) et/ou par équipe.
 - **GitOps** : tous les manifestes versionnés (vu dans l'écosystème).
- “ **The Twelve-Factor App** reste la référence pour les apps cloud-native (config par env, logs vers stdout, processes stateless...). ”

Enrichissement de clusters

Composants indispensables en production

Helm – le gestionnaire de packages



Helm – le gestionnaire de packages

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami/nginx
helm install mon-nginx bitnami/nginx -f values-prod.yaml -n production
helm upgrade mon-nginx bitnami/nginx -f values-prod.yaml
helm rollback mon-nginx 1
helm list -n production
helm uninstall mon-nginx
```

Helm — créer un Chart

```
helm create mon-chart
```

```
mon-chart/
├── Chart.yaml           # Nom, version, description, dépendances
├── values.yaml          # Valeurs par défaut
├── templates/
│   ├── deployment.yaml  # Template avec {{ .Values.xxx }}
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   ├── _helpers.tpl     # Fonctions réutilisables
│   ├── NOTES.txt        # Message post-install
│   └── tests/           # helm test
└── charts/              # Sous-charts (dépendances)
```

Helm – créer un Chart

```
# extrait de templates/deployment.yaml
replicas: {{ .Values.replicaCount }}
image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
resources:
  {{- toYaml .Values.resources | nindent 12 }}
  {{- if .Values.autoscaling.enabled }}
# HPA sera créé séparément
  {{- end }}
```

Helm – gabarits et fonctions

```
# Conditions
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: {{ include "mon-chart.fullname" . }} }
{{- end }}

# Boucles
env:
{{- range $key, $val := .Values.env }}
  - name: {{ $key }}
    value: {{ $val | quote }}
{{- end }}

# Inclusions et helpers
metadata:
  labels: {{- include "mon-chart.labels" . | nindent 4 }}

# Fonctions
{{ .Values.image.tag | default .Chart.AppVersion }}
{{ .Release.Name | trunc 63 | trimSuffix "-" }}
{{ now | date "2006-01-02" }}
{{ randAlphaNum 16 | b64enc }}

# Validation et debug
helm lint ./mon-chart
helm template mon-chart ./mon-chart --values values-prod.yaml
helm install mon-chart ./mon-chart --dry-run --debug
```

Metrics Server

API agrégée qui collecte les métriques **CPU/RAM** des Pods et des Nodes via le kubelet.

```
# Installation (Kind / on-premise)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Patch indispensable en Kind / dev (certificats kubelet self-signed)
kubectl patch deployment metrics-server -n kube-system --type='json' \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
```



```
# Utilisation
kubectl top nodes
kubectl top pods -A
kubectl top pods --sort-by=memory -n dev
```

Metrics Server

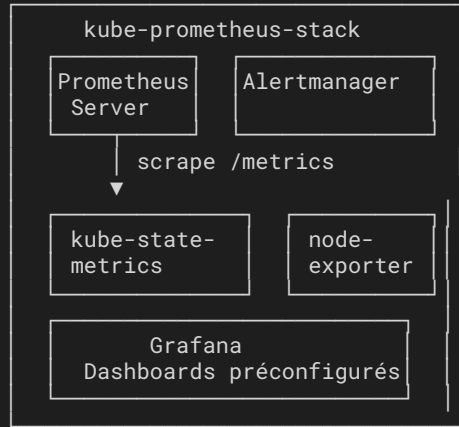
Pourquoi c'est essentiel :

- Requis par le **HPA** (autoscaling sur CPU/RAM)
 - Outils comme **k9s**, dashboards Lens en dépendent
 - Visibilité de base sur la consommation
- “ Metrics Server n'est **pas** une solution de monitoring complète — il ne stocke pas l'historique. Pour ça → Prometheus. ”

Prometheus + Grafana

Prometheus : collecte de métriques par scraping HTTP, stockage time-series, langage PromQL.

Grafana : dashboards visuels, alerting.



Installation : kube-prometheus-stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  --set grafana.adminPassword=admin \
  --set prometheus.prometheusSpec.retention=7d

# Accès Grafana
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
# http://localhost:3000 (admin / admin)
```

Installation : kube-prometheus-stack

Le chart installe :

- Prometheus Server + Operator
- Alertmanager
- Grafana avec dashboards Kubernetes pré-importés
- kube-state-metrics (métriques sur les objets K8s)
- node-exporter (métriques système des nœuds)
- CRDs : ServiceMonitor, PodMonitor, PrometheusRule

ServiceMonitor — scraper sa propre app

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api-monitor
  namespace: prod
  labels: { release: monitoring }      # matché par le Prometheus du chart
spec:
  selector:
    matchLabels: { app: api }
  endpoints:
    - port: metrics                    # nom du port dans le Service
      interval: 30s
      path: /metrics
```

ServiceMonitor — scraper sa propre app

PrometheusRule — alertes :

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: api-alerts
spec:
  groups:
    - name: api.rules
      rules:
        - alert: HighErrorRate
          expr: |
            sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
              / sum(rate(http_requests_total[5m])) by (service) > 0.05
          for: 10m
          labels: { severity: warning }
          annotations:
            summary: "Taux d'erreur > 5% sur {{ $labels.service }}"
```

PromQL – exemples utiles

```
# CPU utilisé par pod (cœurs)
sum(rate(container_cpu_usage_seconds_total{namespace="prod"}[5m])) by (pod)

# Mémoire par pod (Mio)
sum(container_memory_working_set_bytes{namespace="prod"}) by (pod) / 1024 / 1024

# Pods qui crashloopen depuis 10 min
kube_pod_container_status_waiting_reason{reason="CrashLoopBackOff"} == 1

# Latence p99 d'un endpoint
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))

# Disponibilité d'un Deployment
kube_deployment_status_replicas_available
  / kube_deployment_spec_replicas

# Top 10 pods par mémoire
topk(10, sum(container_memory_working_set_bytes) by (pod, namespace))
```

cert-manager + Let's Encrypt

cert-manager : Operator qui automatise l'émission et le renouvellement de certificats TLS.

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true
```

cert-manager + Let's Encrypt

ClusterIssuer Let's Encrypt (challenge HTTP-01) :

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: letsencrypt-prod }
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@formation.local
    privateKeySecretRef: { name: letsencrypt-prod-key }
    solvers:
      - http01:
          ingress:
            class: nginx
```

“ Pour Let's Encrypt en prod, votre Ingress doit être accessible publiquement sur 80/443. ”

Certificat automatique sur un Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts: [app.formation.local]
      secretName: app-tls          # créé automatiquement
  rules:
    - host: app.formation.local
      http:
        paths:
          - { path: /, pathType: Prefix, backend: { service: { name: app, port: { number: 80 } } } }
```

Certificat automatique sur un Ingress

Vérifier :

```
kubectl get certificate -A  
kubectl describe certificate app-tls  
kubectl get certificaterequest -A  
kubectl get order -A          # ressources ACME
```

“ Renouvellement automatique 15 jours avant expiration (défaut).
Aucune intervention humaine. ”

EFK – Logs centralisés

EFK = Elasticsearch + Fluent Bit (ou Fluentd) + Kibana.



EFK — Logs centralisés

Alternatives modernes :

- **Loki + Promtail + Grafana** (PLG) — plus léger, intégré à Grafana
- **OpenSearch** (fork open de Elasticsearch) + Fluent Bit + OpenSearch Dashboards

EFK — installation rapide

```
# Namespace
kubectl create namespace logging

# Elasticsearch (mode single-node pour le lab)
helm repo add elastic https://helm.elastic.co
helm install elasticsearch elastic/elasticsearch -n logging \
  --set replicas=1 --set minimumMasterNodes=1 \
  --set resources.requests.memory=1Gi

# Kibana
helm install kibana elastic/kibana -n logging

# Fluent Bit (DaemonSet)
helm repo add fluent https://fluent.github.io/helm-charts
helm install fluent-bit fluent/fluent-bit -n logging \
  --set config.outputs="[OUTPUT]"
  Name es
  Match *
  Host elasticsearch-master
  Port 9200
  Index k8s
  Suppress_Type_Name On"

# Accès Kibana
kubectl port-forward -n logging svc/kibana-kibana 5601:5601
# http://localhost:5601
```

Fluent Bit — pipeline de logs

```
[INPUT]
  Name tail
  Path /var/log/containers/*.log
  Parser cri
  Tag kube.*
  Refresh_Interval 5

[FILTER]
  Name kubernetes
  Match kube.*
  Kube_URL https://kubernetes.default.svc:443
  Merge_Log On
  K8S-Logging.Parser On
  K8S-Logging.Exclude Off
  Annotations Off

[FILTER]
  Name modify
  Match *
  Add cluster formation-k8s

[OUTPUT]
  Name es
  Match *
  Host elasticsearch-master.logging.svc
  Port 9200
  Logstash_Format On
  Logstash_Prefix k8s
  Retry_Limit 5
```

“ Le filtre **kubernetes** enrichit automatiquement chaque log avec : namespace, pod, container, labels, annotations. ”

Service Mesh avec Istio

Réseau intelligent pour microservices

Architecture d'Istio



Sidecar Envoy : injecté automatiquement dans chaque Pod. Intercepte tout le trafic.

Composants Istio

Composant	Rôle
istiod	Control plane unifié : Pilot (config Envoy), Citadel (certificats), Galley (validation)
Envoy (sidecar)	Proxy L7 dans chaque Pod : routing, retry, timeout, circuit breaker, métriques
Istio Gateway	Envoy en bordure : ingress/egress du mesh (remplace Ingress)
Kiali	UI graphique du mesh (topologie, trafic)
Jaeger / Zipkin	Tracing distribué
Prometheus	Métriques collectées par Envoy

Composants Istio

```
# Installation
curl -L https://istio.io/downloadIstio | sh -
cd istio-*
export PATH=$PWD/bin:$PATH

istioctl install --set profile=demo -y

# Activer l'injection automatique sur un namespace
kubectl label namespace prod istio-injection=enabled

# Addons (Kiali, Jaeger, Grafana, Prometheus)
kubectl apply -f samples/addons
```

Ressources Istio clés

Gateway — entrée du mesh :

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata: { name: app-gw }
spec:
  selector: { istio: ingressgateway }
  servers:
    - port: { number: 80, name: http, protocol: HTTP }
      hosts: [app.formation.local]
```

Ressources Istio clés

VirtualService — règles de routage :

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: app-vs }
spec:
  hosts: [app.formation.local]
  gateways: [app-gw]
  http:
    - match: [{ uri: { prefix: /api } }]
      route:
        - destination: { host: api-svc, port: { number: 8080 } }
    - route:
      - destination: { host: frontend-svc, port: { number: 80 } }
```

DestinationRule et subsets

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata: { name: api-dr }
spec:
  host: api-svc
  trafficPolicy:
    connectionPool:
      tcp: { maxConnections: 100 }
      http: { http2MaxRequests: 1000 }
    outlierDetection: # circuit breaker
      consecutive5xxErrors: 5
      interval: 30s
      baseEjectionTime: 60s
  subsets:
    - name: v1
      labels: { version: v1 }
    - name: v2
      labels: { version: v2 }
```

“ Le `DestinationRule` définit des **subsets** (groupes de pods par labels) que le `VirtualService` peut référencer pour le routage avancé. ”

Observabilité — Kiali

Kiali offre une vue graphique en temps réel du mesh.

- **Graphe de service** : qui parle à qui, débit, erreurs
- **Traces distribuées** : intégration Jaeger
- **Configuration** : valider VirtualService, DestinationRule, Gateway
- **Métriques** : intégration Prometheus

```
# Accès Kiali (après installation des addons)
istioctl dashboard kiali
# http://localhost:20001
```

Observabilité — Kiali

Tracing distribué (Jaeger) :

- Chaque requête traversant le mesh est tracée
- Les headers `x-b3-traceid`, `x-b3-spanid` sont propagés
- L'application doit **propager** ces headers entre ses appels sortants (sinon, traces incomplètes)

```
istioctl dashboard jaeger
```

Blue/Green avec Istio

Bascule de trafic atomique entre deux versions :

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: api-vs }
spec:
  hosts: [api-svc]
  http:
    - route:
      - destination: { host: api-svc, subset: v1 }
        weight: 100 # 100% sur v1 (blue)
      - destination: { host: api-svc, subset: v2 }
        weight: 0 # 0% sur v2 (green)
```

Blue/Green avec Istio

Bascule (1 commande, atomique) :

```
kubectl patch virtualservice api-vs --type=merge -p '
spec:
  http:
    - route:
      - { destination: { host: api-svc, subset: v1 }, weight: 0 }
      - { destination: { host: api-svc, subset: v2 }, weight: 100 }
'
```

“ Plus puissant qu'un Service kubectl patch : on contrôle **par % de requêtes**, pas par nombre de pods. ”

Canary progressif et routage avancé

10% → 50% → 100% sans redéploiement :

```
spec:
  http:
    - route:
      - { destination: { host: api-svc, subset: v1 }, weight: 90 }
      - { destination: { host: api-svc, subset: v2 }, weight: 10 }
```

Canary progressif et routage avancé

Canary par header (interne only) :

```
spec:
  http:
    - match:
        - headers:
            x-beta-tester: { exact: "true" }
      route:
        - destination: { host: api-svc, subset: v2 }
    - route:
        - destination: { host: api-svc, subset: v1 }
```

Canary progressif et routage avancé

Injection de fautes pour tester la résilience :

```
spec:
  http:
    - fault:
        delay: { percentage: { value: 10 }, fixedDelay: 5s }
        abort: { percentage: { value: 1 }, httpStatus: 503 }
    route:
      - destination: { host: api-svc }
```

Mutual TLS (mTLS)

Istio chiffre automatiquement tout le trafic intra-mesh avec **mTLS**.

```
# PeerAuthentication – exiger mTLS dans un namespace
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: { name: default, namespace: prod }
spec:
  mtls:
    mode: STRICT          # PERMISSIVE (mixte), DISABLE
```

Mutual TLS (mTLS)

Modes :

Mode	Comportement
PERMISSIVE	Accepte mTLS et plaintext (par défaut, transition douce)
STRICT	Refuse tout trafic non-mTLS (production cible)
DISABLE	Désactive complètement mTLS

Vérification :

```
istioctl authn tls-check api-svc.prod.svc.cluster.local
```

AuthorizationPolicy

Politique L7 d'autorisation entre services.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: api-allow-frontend, namespace: prod }
spec:
  selector:
    matchLabels: { app: api }
  action: ALLOW
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/prod/sa/frontend"]
      to:
        - operation:
            methods: [GET, POST]
            paths: [/api/*]
```

AuthorizationPolicy

Bonnes pratiques mTLS :

1. Commencer en `PERMISSIVE` pour migrer sans casse
2. Auditer avec `istioctl authn tls-check`
3. Passer en `STRICT` namespace par namespace
4. Ajouter des `AuthorizationPolicy deny-all` puis whitelister
5. Utiliser `principals` (identité SA) plutôt qu'IP

Écosystème de Kubernetes

La galaxie cloud-native

CNCF Landscape

La **CNCF** (Cloud Native Computing Foundation) référence > 1 700 projets cloud-native.

CNCF Landscape

```
Orchestration : Kubernetes
Service Mesh  : Istio, Linkerd, Cilium
Observability : Prometheus, Jaeger, OTel
GitOps        : ArgoCD, Flux
CI/CD         : Tekton, Argo Workflows
Security      : Falco, OPA, Kyverno
Storage       : Rook, Longhorn
Networking    : Calico, Cilium, Envoy
Packaging     : Helm, Kustomize
Runtime       : containerd, CRI-O
DBaaS        : Vitess, TiKV, etcd
```

CNCF Landscape

Niveaux de maturité CNCF :

- **Graduated** : production-ready, écosystème mature (K8s, Prometheus, Istio, Helm, ArgoCD...)
- **Incubating** : en croissance, adoption réelle (Cilium, OpenTelemetry, KEDA...)
- **Sandbox** : early stage, à surveiller

Operators — étendre Kubernetes

Un **Operator** = controller + CRD pour gérer une app complexe.

```
Custom Resource → Controller → Actions K8s  
(état désiré)      (boucle de      (créer pods, PVC,  
                    réconciliation) services...)
```

Operators – étendre Kubernetes

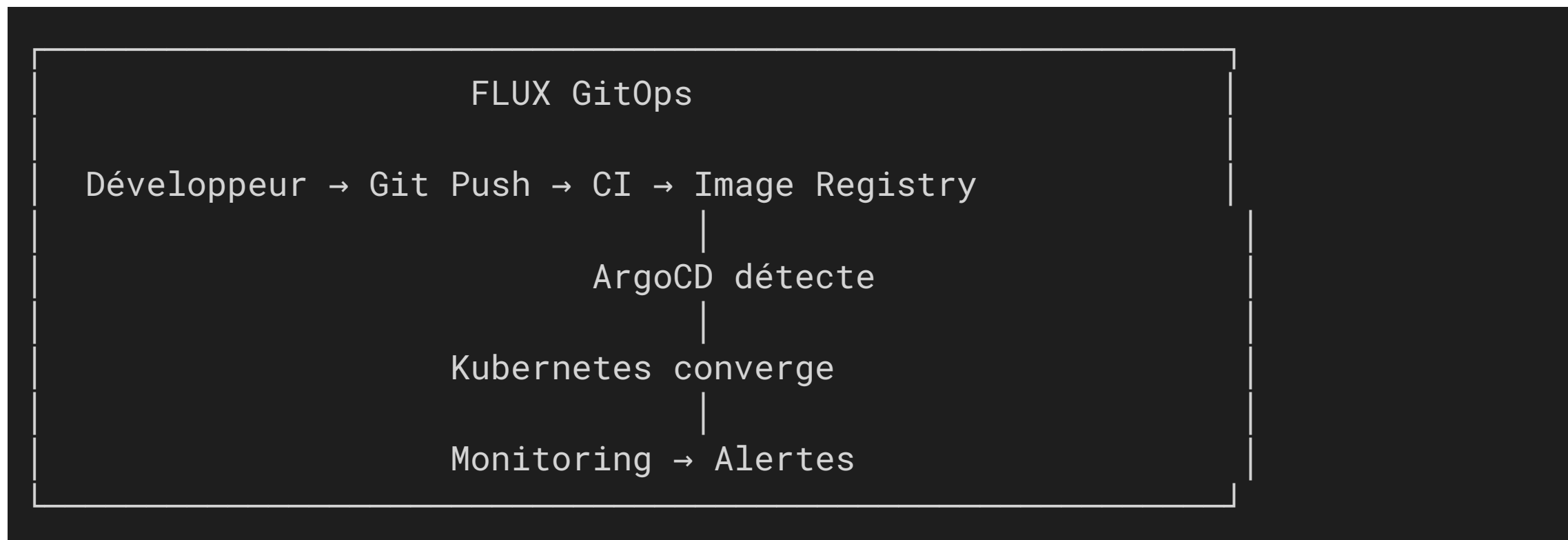
Operator	Gère
cert-manager	Certificats TLS (Let's Encrypt, Vault, CA interne)
Prometheus Operator	Prometheus, Alertmanager, ServiceMonitor
ArgoCD	Applications GitOps
Strimzi	Clusters Kafka complets (brokers, Zookeeper, KafkaConnect)
Zalando Postgres	Clusters PostgreSQL HA avec failover
MongoDB Community	ReplicaSets MongoDB
Velero	Backup/restore du cluster
External Secrets	Synchronise des secrets depuis Vault, AWS SM, GCP SM

“ **OperatorHub.io** : catalogue d'Operators prêts à l'emploi.

”

GitOps — ArgoCD et Flux

Principe : Git = source de vérité unique. Un controller dans le cluster applique en permanence ce qui est dans Git.



GitOps – ArgoCD et Flux

Outil	Caractéristiques
ArgoCD	UI riche, multi-cluster, RBAC, sync-waves, ApplicationSet
Flux	CLI-first, GitOps Toolkit modulaire, soutenu par Weaveworks/CNCF
Argo Rollouts	Déploiements progressifs (Canary, Blue/Green) automatisés

Sécurité avancée – Policy as Code

Outil	Rôle
OPA Gatekeeper	Admission policies en Rego (OPA) – interdire <code>:latest</code> , exiger des labels, etc.
Kyverno	Admission policies en YAML (plus simple qu'OPA)
Falco	Détection d'anomalies runtime (syscalls, fichiers, réseau)
Trivy / Grype	Scan de vulnérabilités d'images
Cosign	Signature et vérification d'images (Sigstore)
kube-bench	Audit CIS Benchmark
kube-hunter	Pentest passif du cluster

Sécurité avancée — Policy as Code

```
# Exemple Kyverno : interdire :latest
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: disallow-latest-tag }
spec:
  validationFailureAction: Enforce
  rules:
    - name: require-image-tag
      match: { resources: { kinds: [Pod] } }
      validate:
        message: "Image tag :latest interdit"
        pattern:
          spec:
            containers:
              - image: "!*:latest"
```

CI/CD natif Kubernetes

Outil	Style
Tekton	Pipelines déclaratifs (CRDs Task, Pipeline, PipelineRun)
Argo Workflows	Workflows DAG (data pipelines, batch)
Jenkins X	Jenkins repensé pour Kubernetes
GitLab Runner	Runner Kubernetes natif (auto-scaling)
GitHub Actions self-hosted	Runners dans le cluster
Spinnaker	Déploiements multi-cloud complexes

CI/CD natif Kubernetes

```
# Exemple Tekton - pipeline minimal
apiVersion: tekton.dev/v1
kind: Pipeline
metadata: { name: build-deploy }
spec:
  params:
    - { name: image, type: string }
  tasks:
    - name: build
      taskRef: { name: kaniko }
      params: [{ name: IMAGE, value: $(params.image) }]
    - name: deploy
      runAfter: [build]
      taskRef: { name: kubectl-apply }
```

Service Mesh – alternatives à Istio

Mesh	Caractéristiques
Linkerd	Plus léger qu'Istio, en Rust, configuration simple, CNCF Graduated
Cilium Service Mesh	Sans sidecar (basé sur eBPF), intégré au CNI Cilium
Consul Connect	HashiCorp, fonctionne aussi hors-Kubernetes
Kuma	Multi-cluster, multi-zone, par Kong
AWS App Mesh	Managé AWS (en perte de vitesse, AWS recommande Istio sur EKS)

Service Mesh – alternatives à Istio

Comparaison rapide :

Critère	Istio	Linkerd	Cilium
Complexité	élevée	faible	moyenne
Empreinte	lourde (sidecar)	légère (sidecar Rust)	très légère (eBPF)
Fonctionnalités	toutes	essentielles	en croissance
Multi-cluster	oui	oui	oui